HTTP 5주차

엔터티와 인코딩

본문은 어떻게 실려오는가

출처:「HTTP 완벽 가이드」15장 엔터티와 인코딩

HTTP 메시지 vs 엔터티

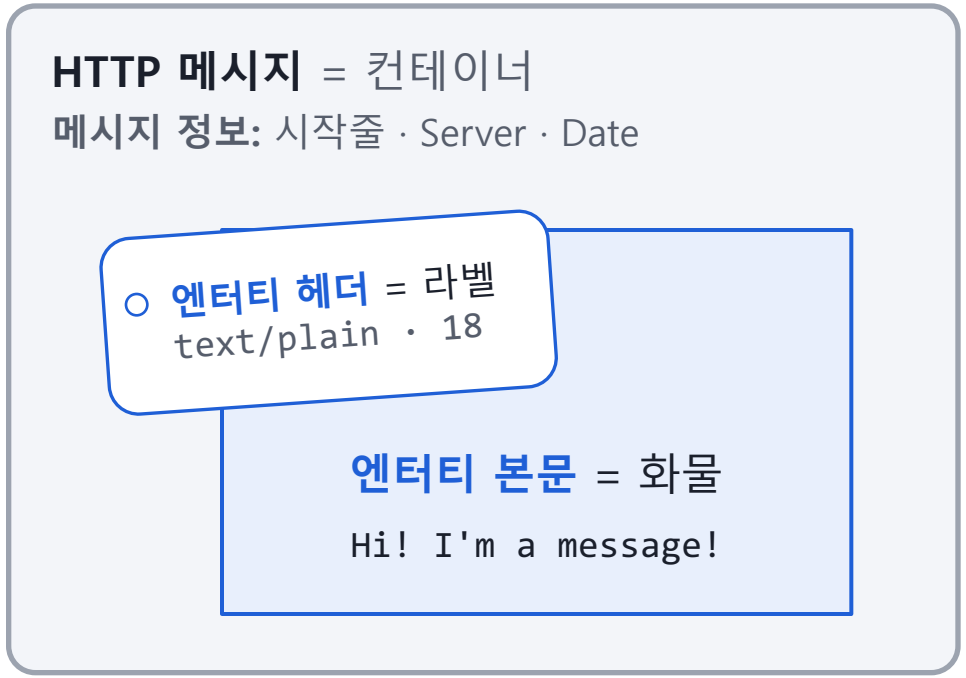
```
HTTP/1.0 200 OK
Server: Netscape-Enterprise/3.6
Date: Sun, 17 Sep 2000 00:01:05 GMT
Content-type: text/plain
Content-length: 18
Hi! I'm a message!
```

엔터티

메시지 정보

엔터티 헤더

엔터티 본문



HTTP 메시지는 화물을 실어 나르는 **컨테이너**, 엔터티는 그 안에 담긴 **실제 화물**이다.

엔터티 헤더 필드

콘텐츠 정보 무엇인가

Content-Type
본문 종류 (예: text/html, application/json)

Content-Length
본문의 크기 (바이트)

Content-Language
본문의 자연어 (예: ko, en)

Content-Encoding
적용된 변형 (예: gzip 압축)

Content-Location
이 콘텐츠의 또 다른 위치(URL)

Content-Range
부분 전송 시 전체 중 어느 부분인지

Content-MD5
본문 무결성 확인용 체크섬

캐시·신선도 언제까지

Last-Modified
마지막 생성·수정 시각

Expires
신선하지 않게 되는 시각

Cache-Control*
캐시 방법에 대한 지시

기타 그 밖의 정보

Allow
허용 메서드 (예: GET, HEAD)

ETag*
이 버전의 고유 식별자

엔터티 헤더는 본문(화물)에 붙은 **라벨**로, 데이터가 무엇이고 어떻게 다뤄야 하는지 알려준다.

* ETag, Cache-Control은 공식적으로 엔터티 헤더는 아니지만, 엔터티와 밀접하게 함께 쓰인다.

Content-Length: 본문의 끝을 알려주는 헤더

엔터티 본문의 크기를 **바이트** 단위로 표시한다. 글자 수가 아니다: UTF-8 "가나다" = **9바이트**

크기를 알 때

```
HTTP/1.1 200 OK
Content-Type: text/plain
Content-Length: 18
Hi! I'm a message!      ← 본문 18바이트
```

크기를 모를 때

```
HTTP/1.1 200 OK
Content-Type: text/plain
Transfer-Encoding: chunked   ← Content-Length 없음
7    ← 다음 7바이트
Hello,
6    ← 다음 6바이트
world!
0    ← 크기 0 = 끝
```



지속 커넥션
다음 메시지와 경계 구분 (필수)



잘림 감지
받은 크기와 비교해 끊김 확인



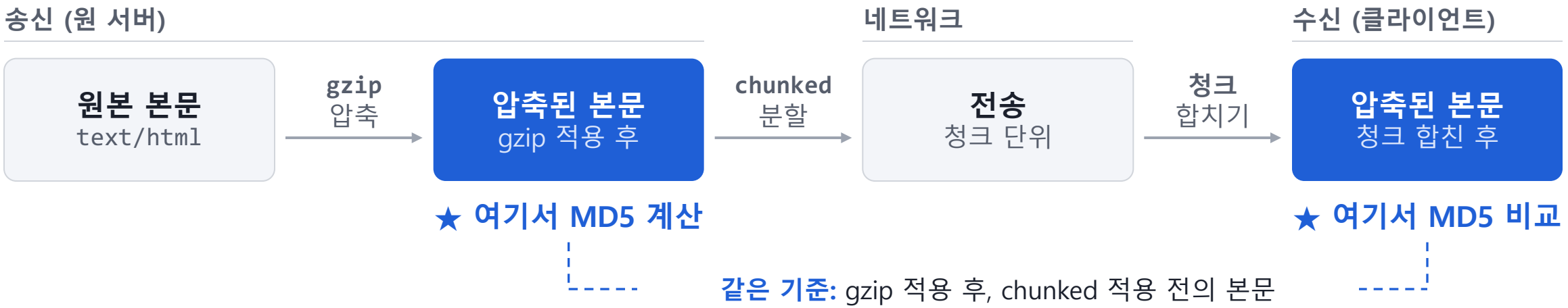
요청 본문
연결을 끊어 끝을 알릴 수 없음

> 크기를 알면 **Content-Length**, 모르면 **chunked**. **둘 중 하나만**

둘을 함께 쓰면 서버마다 본문 끝을 다르게 해석해 HTTP Request Smuggling 취약점의 원인이 될 수 있다.

Content-MD5: 본문이 변하지 않았는지 확인하기

TCP가 신뢰할 수 있어도 **중간 프록시가 본문을 망가뜨릴 수 있다.** 체크섬으로 **종단 간 무결성**을 확인한다.



규칙

원 서버만 계산해서 헤더에 넣는다

프록시·캐시는 **추가·수정 금지**

바꾸면 종단 간 검증 목적이 깨진다

활용과 확장

해시 테이블 키: 문서 위치 탐색, 중복 저장 방지

Want-Digest: 원하는 알고리즘과 선호도(q) 지정

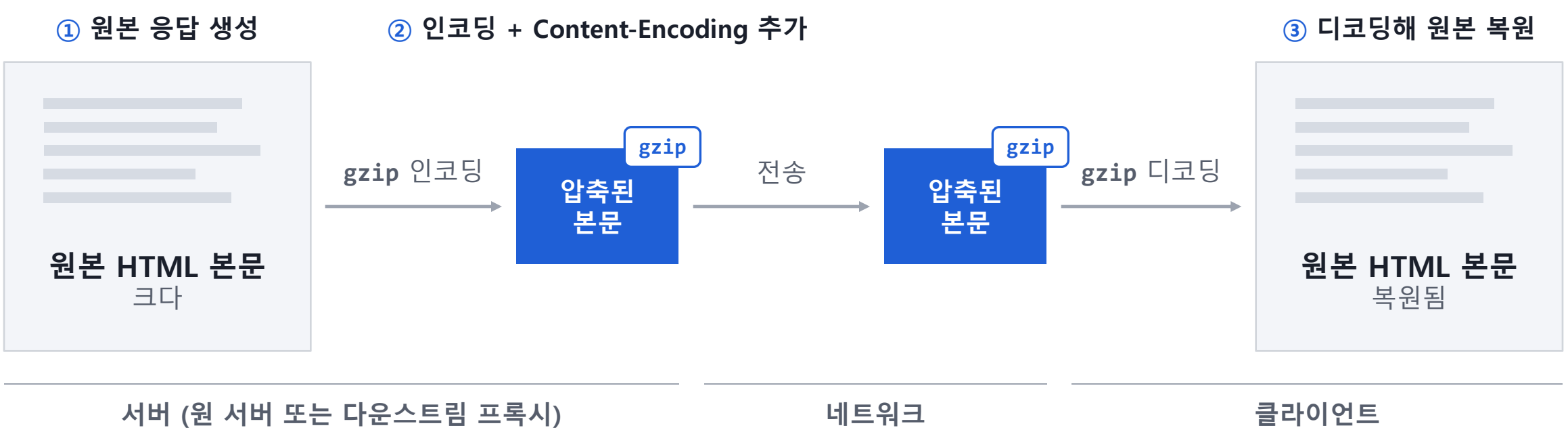
다만 실제로는 자주 쓰이지 않는다

콘텐츠 인코딩: 보내기 전에 포장하기

보내는 쪽이 전송 전에 콘텐츠를 **변형**하고, 인코딩된 데이터가 그대로 엔터티 본문에 담겨 전송된다.

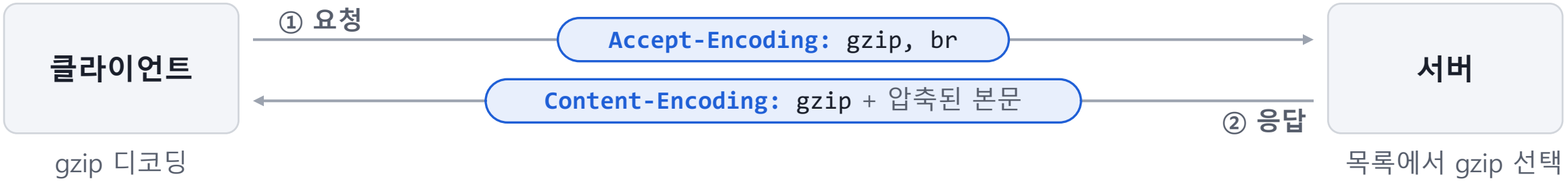
압축 큰 HTML을 느린 클라이언트에게 빠르게 (대표적)

암호화·뒤섞기 제3자가 내용을 볼 수 없도록



인코딩 후 `Content-Type`은 그대로, `Content-Length`는 **인코딩된 크기로 바뀐다**

Accept-Encoding: 내가 풀 수 있는 인코딩 알려주기



예시	의미
`Accept-Encoding: compress, gzip`	compress와 gzip 둘 다 가능
`Accept-Encoding:` (빈 값)	인코딩 안 된 원본(identity)만
`Accept-Encoding: *`	어떤 인코딩이든 가능
`Accept-Encoding: compress;q=0.5, gzip;q=1.0`	둘 다 되지만 gzip을 더 선호
`Accept-Encoding: gzip;q=1.0, identity;q=0.5, *;q=0`	gzip 최우선, 안 되면 원본, 그 외 거부

q값 읽는 법 높을수록 선호 (0.0~1.0) · 생략하면 1.0 · `q=0`이면 거부
identity = 인코딩하지 않은 원본 · 헤더가 없으면 **Accept-Encoding: *** 와 같다

인코딩된 응답은 어떻게 생겼을까?

```
HTTP/1.1 200 OK
Date: Fri, 05 Nov 1999 22:35:15 GMT
Server: Apache/1.2.4
Content-Length: 6096
Content-Type: image/gif
Content-Encoding: gzip
(압축된 바이너리 데이터)
```

인코딩된(압축된) 본문의 크기

원래 형식.
디코딩 후 어떻게 보여줄지 알려줌

인코딩 방식. 이걸 보고 디코딩

헤더	인코딩 전	인코딩 후
Content-Type	image/gif	image/gif (그대로)
Content-Length	원본 크기	**압축된 크기** (바뀜)
Content-Encoding	없음	**gzip** (추가됨)

주의 인코딩 후에도 **Content-Type**은 반드시 있어야 한다. 디코딩 결과를 어떻게 보여줄지 알려주기 때문.

보내는 쪽은 본문을 변형해 보내고 받는 쪽은 되돌린다.
Content-Type = **원래 형식**, **Content-Length** = **인코딩된 크기**

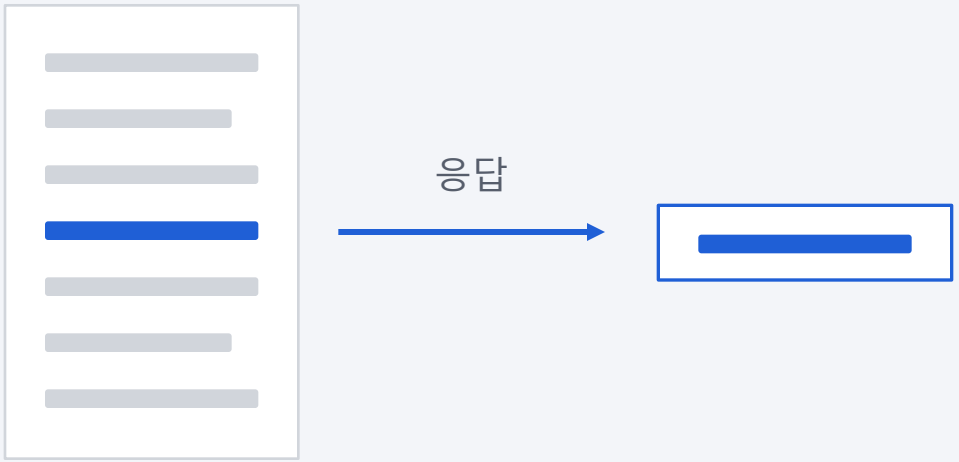
페이지가 조금만 바뀌어도 서버는 문서 전체를 다시 보낸다

기존 방식



바뀐 곳은 한 줄인데 페이지 전체를 받음

델타 인코딩



바뀐 한 줄만 받아 옛 버전에 적용

클라이언트가 가진 옛 버전(인스턴스)을 기준으로 차이를 계산하므로, 델타 인코딩은 **인스턴스 조작**의 일종이다.

클라이언트가 가진 버전을 알리면 서버는 차이만 돌려준다



요청은 헤더 2개로 델타를 청하고, 응답은 226과 헤더 3개로 답한다

요청 (클라이언트 → 서버)

```
GET /news.html
If-None-Match: "v1"
A-IM: diffe
```

- **If-None-Match** 내가 가진 버전의 ETag
- **A-IM** 델타 수용 의사 + 알고리즘

응답 (서버 → 클라이언트)

```
HTTP/1.1 226 IM Used
IM: diffe
ETag: "v2"
Delta-Base: "v1"
```

- **226** 전체 대신 델타를 보낸다
- **IM** 델타 계산에 쓴 알고리즘
- **ETag** 델타 적용 후 생길 새 버전
- **Delta-Base** 기준이 된 옛 버전의 ETag

델타 인코딩은 전송량을 줄이지만, 서버가 옛 버전을 보관해야 한다

	기존 방식 (If-None-Match만)	델타 인코딩
요청 헤더	If-None-Match	If-None-Match + A-IM
문서가 그대로일 때	304 Not Modified	304 Not Modified
문서가 바뀌었을 때	200 OK + 문서 전체	**226 IM Used + 델타**
전송량	문서 전체 크기	**바뀐 부분만**
서버 부담	최신 버전만 보관	옛 버전 보관 + 델타 계산
실제 사용	웹 전반에서 사용	거의 쓰이지 않음

같은 아이디어, 다른 곳에서 Git diff(커밋 간 차이만 저장·전송), 앱 패치 업데이트(바뀐 부분만 내려받기)

한 줄 요약

> 델타 인코딩은 가진 버전을 알려 주고,
> 바뀐 부분만 받는 HTTP 확장이다.